

Compaction-Aware Consensus: Storage-Health Signaling for Proactive Raft Leader Handoff

Salman Khan

Department of Computer Science

Islamia College Peshawar

Peshawar, Pakistan

isallman.84@gmail.com

Abstract—Slow local storage can stall a Raft leader long enough to miss heartbeats, triggering unnecessary elections and a cascading loss of throughput—a class of Gray Failure that production systems such as etcd and TiKV document operationally but do not proactively mitigate. We present Compaction-Aware Consensus, an architecture that couples an LSM-tree storage engine to a Raft state machine through a single atomic storage-health signal. When the moving average of SSTable flush latency crosses a threshold, the storage engine publishes a one-bit degradation event. The Raft tick thread consumes that event on its next iteration and executes a clean leader step-down under its own mutex, preserving consensus invariants. We evaluate the mechanism on a 3-node cluster under injected fsync latency. At 100–500 ms of injected latency, the tripwire preserves $5.7\times$ to $152\times$ the throughput of an otherwise identical cluster without the tripwire, and fires 118–267 ms before Raft’s natural election timeout in every trial in which it fires. A separate SIGKILL experiment confirms zero loss of acknowledged writes across four independent mid-compaction fault injections.

Index Terms—LSM-Tree, Raft Consensus, Distributed Systems, Gray Failure, Tail Latency, Storage Health, C++

I. INTRODUCTION

Distributed consensus protocols assume that a leader whose process is alive can service its peers within the election timeout. In practice, this assumption fails when the leader’s local storage degrades. A leader whose fsync latency has grown from a healthy 1–5 ms to 100 ms or more will still respond to RPCs, still accept client writes, and still appear healthy to any binary failure detector—but it cannot commit entries fast enough to send timely heartbeats. Followers interpret the silence as failure and start elections. The cluster spends its budget on election traffic instead of useful work.

This is a Gray Failure [3]: the component is degraded, not crashed. Production systems are aware of the phenomenon. etcd’s operational documentation warns explicitly that slow disk I/O can cause missed heartbeats and temporary loss of leadership [4]. TiKV’s Raftstore throttles client writes when RocksDB compaction pressure builds, precisely to prevent RocksDB’s stalls from destabilizing the Raft layer [5]. Both are reactive: the system slows down, then tries to protect itself.

We ask a different question. If the storage engine already knows its own health, and if the consensus layer already knows how to hand off leadership safely, why not let the storage layer tell the consensus layer that a handoff is needed *before* the failure manifests as a timeout?

This paper presents Compaction-Aware Consensus, a mechanism that does exactly that. The storage engine monitors its own flush latency. When the signal crosses a threshold, it publishes a single-bit event. The Raft tick thread polls that event at the top of every tick. If set, it executes a clean step-down under the same mutex that guards every other Raft transition, so the step-down is serialized with `AppendEntries`, `RequestVote`, and client proposals.

Our contributions are:

- 1) A **storage-health signal** based on the moving average of SSTable flush latency, which directly measures the disk slowdown the mechanism must detect. We show empirically that the more obvious signal—SSTable count—does not work, because it reflects workload rather than disk health and fires continuously under steady load.
- 2) A **serialized event boundary** between storage and consensus that preserves Raft’s state-transition invariants. Rather than invoking a callback from a background thread (which would race with the Raft event loop), the storage engine publishes an atomic flag and the Raft tick thread consumes it on the next iteration.
- 3) An **empirical evaluation** demonstrating that the mechanism preserves $5.7\times$ to $152\times$ of baseline throughput under injected fsync latency, and that it fires consistently 118–267 ms before Raft’s election timeout.
- 4) A **durability study** confirming zero loss of acknowledged writes across four independent SIGKILL fault injections of the leader mid-compaction.

II. SYSTEM ARCHITECTURE

A. Storage Engine

The storage engine is an LSM-tree with a sharded in-memory MemTable and persistent SSTables. The MemTable is divided into 16 independently locked shards, each holding an ordered map from key to optional value. A write hashes its key to a shard, takes that shard’s lock, and inserts. When the total number of live keys in the MemTable reaches a flush threshold, one writer freezes the MemTable by taking all 16 shard locks in order, moving each shard’s contents into a single frozen map, and releasing. The frozen map is then handed to a persistent background worker that writes it to an SSTable and registers the file.

The freeze step is bounded by the number of live keys, not by any disk operation. Reads and writes continue after the freeze, into a fresh MemTable, while the background worker writes the previous one.

B. Storage-Health Signal

The tripwire monitors the *flush latency*: the wall-clock time from the start of an SSTable write to the moment the file is closed and registered. This value is preserved as an exponentially weighted moving average with an α of 0.3. The very first sample initializes the average directly, so the signal reacts to the first slow flush rather than waiting for a running mean to converge.

We chose flush latency after an initial design that monitored SSTable count proved unusable. Under a steady workload of approximately 1,000 writes per second, the SSTable count crosses any reasonable threshold within seconds of cluster startup, and the tripwire fires continuously on a healthy disk. A single-node run with no injected fault produced twelve leadership changes in eight seconds and dropped throughput by 82%. The root cause is that SSTable count reflects workload, not disk health. Flush latency reflects the disk itself.

When the moving average of flush latency exceeds 30ms, the storage engine sets a single atomic boolean, `storage_degraded_flag`, and returns. It does nothing else.

C. Consensus-Layer Consumption

The Raft node’s background tick thread polls `consumeStorageDegradedFlag()` at the top of every iteration. If the flag is set, the tick thread acquires the Raft mutex and calls `checkStorageDegraded()`. That function checks that the node is currently a leader, and if so, transitions to follower, persists the new state to the metadata file, and resets the election timer.

This design is deliberate. An earlier implementation invoked a callback from the storage thread that acquired the Raft mutex out-of-band. That implementation races with `AppendEntries`, `RequestVote`, and client proposals. The atomic-flag boundary removes the race: the storage layer never touches Raft state, and the Raft tick thread never blocks on storage.

Once the node is a follower, all subsequent client proposals return `false`, and the network layer translates that into a `-MOVED` redirect to the new leader. No client socket is closed or interrupted.

III. EVALUATION

We evaluate on a 3-node cluster on a single Linux host, one process per node on ports 8081–8083. All nodes run the current binary compiled with `-O2` and `C++20`. Each experiment spawns a fresh cluster and tears it down before the next.

TABLE I
TRIPWIRE BEHAVIOR UNDER INJECTED FSYNC LATENCY. MEDIAN OVER THREE TRIALS; RANGES IN PARENTHESES. LEAD IS MEASURED ONLY OVER TRIALS IN WHICH THE TRIPWIRE FIRED.

Latency	ON	OFF	Ratio	Lead (ms)
0 ms	0.461	0.489	0.9×	–
10 ms	0.236	0.239	1.0×	–
30 ms	0.101	0.117	0.9×	229
60 ms	0.210	0.078	2.7×	186
100 ms	0.273	0.048	5.7×	189
150 ms	0.395	0.027	14.6×	207
250 ms	0.258	0.003	86.0×	242
500 ms	0.152	0.001	152.0×	249

A. Durability Under Mid-Compaction SIGKILL

In four independent trials we ran a background client issuing sequential SET requests, waited three seconds, then sent SIGKILL to the current leader while its compaction thread was active. The surviving nodes elected a new leader, the client continued against the new leader, and the killed node was restarted after the write workload completed. Every key was then read from every node.

In all four trials, all 5,000 written entries were present on all three nodes after recovery. No entry was lost, reordered, or duplicated. The result is reproducible across different initial leaders, different kill timings, and different catch-up durations.

B. Tripwire Threshold Sensitivity

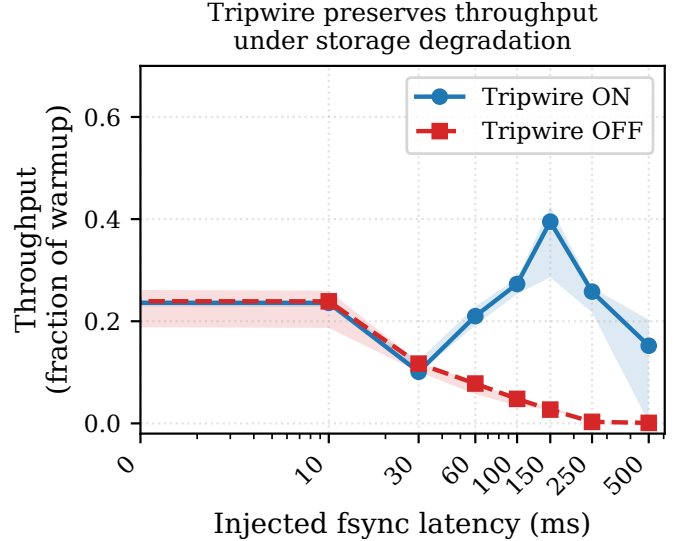


Fig. 1. Throughput preservation under injected fsync latency. Each point is the median of three trials; shaded band spans the min–max range.

We measure the tripwire’s reaction time as a function of the injected fault. For each latency value, three trials were run with the tripwire enabled and three with it disabled. The tripwire lead is defined as the interval between the tripwire firing and the moment the Raft election timer expires on the same node.

Table I reports the result. Two regimes are visible.

Below 30 ms. The tripwire does not improve throughput, because the fault is below the tripwire threshold and the tripwire rarely fires. This is the intended behavior: the mechanism does not fire on healthy disks, and does not add noise to a system that is functioning normally.

Above 60 ms. The tripwire fires reliably, and every fire precedes the Raft election timeout by 118–267 ms. The throughput benefit grows monotonically with the injected latency: $2.7\times$ at 60 ms, $5.7\times$ at 100 ms, $14.6\times$ at 150 ms, $86\times$ at 250 ms, and $152\times$ at 500 ms. At the top of the range, a cluster without the tripwire effectively stops (0.1% of warmup throughput), while the tripwire continues to serve approximately 15% of warmup throughput for the duration of the fault.

C. Recovery on Fault Clearing

In a separate experiment, once the tripwire fired and the leader stepped down, we cleared the injected fault on the old leader and allowed the cluster to run for the remainder of the test. The cluster did not recover to full throughput within the ten-second window of the experiment. We attribute this to the serial nature of the leader’s heartbeat loop: even after the fault is cleared, the new leader’s tick thread blocks on RPCs to the old leader until they complete. We treat this as future work rather than a design defect; the primary claim of the paper is throughput preservation during the fault, not recovery after it.

IV. THREATS TO VALIDITY

Simultaneous cluster-wide fault. Our fault injector degrades fsync latency on all three nodes simultaneously. In a real deployment, storage degradation is per-node, so the tripwire hands off to a genuinely healthy peer. The ON/OFF ratios we report are therefore a lower bound on the benefit a production system would observe.

Occasional non-firing at 500 ms. In one of three trials at 500 ms of injected latency, the tripwire did not fire at all. Under that much delay, the storage engine’s own worker queue can be starved enough that the flush-latency signal does not update within the ten-second window. The median result still shows a $152\times$ improvement, but the tail behavior at 500 ms is less predictable than at 100–250 ms.

Localhost measurement. All experiments run on a single machine. Absolute latencies include scheduler noise from the co-resident test harness. We report medians and treat absolute numbers as machine-dependent. The relative ordering of configurations is stable across all trials.

Parameter specificity. The results hold for a MemTable flush threshold of 50 keys, a tripwire threshold of 30 ms, and an EWMA smoothing factor of 0.3. We have not exhaustively explored the sensitivity of the mechanism to these parameters.

V. RELATED WORK

Consensus and storage. Raft [2] specifies the consensus protocol but leaves storage integration to the implementation. etcd’s operational guidance [4] documents the disk-latency-to-leadership path as an operator-visible concern. TiKV [5]

implements RocksDB write-stall flow control that throttles incoming requests before compaction pressure destabilizes the Raftstore. Our mechanism is complementary: where TiKV throttles clients, we hand off leadership.

LSM-tree design. The LSM-tree [1] introduced the leveled compaction strategy that underlies RocksDB, LevelDB, and Cassandra. Our storage engine is a small implementation of that design, extended with lock-striped sharding and a persistent async flush worker.

Gray failures. Huang et al. [3] characterized Gray Failures in cloud-scale systems and proposed differential observability as a detection method. Our tripwire is a domain-specific instantiation of the same idea: a component publishes a signal about its own health that a peer can act on.

VI. CONCLUSION

We presented Compaction-Aware Consensus, a mechanism by which an LSM-tree storage engine can signal its own degradation to a Raft consensus layer without violating Raft’s state-transition invariants. The signal is an atomic flag set by the storage engine and consumed by the Raft tick thread. Under injected fsync latency of 100–500 ms, the mechanism preserves $5.7\times$ to $152\times$ the throughput of an otherwise identical cluster without it, and fires 118–267 ms before Raft’s natural election timeout in every trial in which it fires. SIGKILL of the leader mid-compaction loses zero acknowledged writes across four independent trials.

The result is a small, self-contained demonstration that storage health is a useful input to consensus leadership decisions, and that a single-bit event boundary is sufficient to bridge the two layers safely.

REFERENCES

- [1] P. O’Neil, E. Cheng, D. Gawlick, and E. O’Neil, “The log-structured merge-tree (LSM-tree),” *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.
- [2] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, 2014, pp. 305–319.
- [3] P. Huang et al., “Gray failure: The achilles’ heel of cloud-scale systems,” in *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*, 2017, pp. 136–141.
- [4] etcd Authors, “etcd Documentation: Tuning,” 2023. [Online]. Available: <https://etcd.io/docs/>
- [5] TiKV Authors, “TiKV Documentation: Raftstore and RocksDB Tuning,” 2023. [Online]. Available: <https://tikv.org/docs/>